MIT

(AnAutonomousInstitute Affiliated to Savitribai Phule PuneUniversity)

Academy of Engineering

Subject (C ode) :

Essentials of Data Science

EDS CodeTantra Programs

Name	PRN	Div
Pranali Sanjay Kale	202501120121	DS2(D22)

1.2LabAssignments

1.2.1. Pass or Fail
1.2.2. Fibonacci Series using Recursive Function 1.2.3.Pattern-1
1.2.4.Pattern-2
2.2. Lab Assignments
2.2.1. Linear Search Technique
2.2.2. Captain of the Team

3.2 Lab Assignments
3.2.1. NumPy: Matrix Operations
3.2.2. NumPy Horizontal and Vertical Stacking of Arrays
3.2.3. NumPy: Custom Sequence Generation
3.2.4. NumPy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise O pe rators
3.2.5. NumPy: Copying and Viewing Array
3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting
3.2.7. Student Data Analysis and Operations

4.2. Lab Assignments

4.2.1. Month with the Highest Total Sales
4.2.2. Best Selling Product
4.2.3. City that Sold the Most Products
4.2.4. Most Frequently Sold Product Pairs
4.2.5. Titanic Dataset Analysis and Data Cleaning
4.2.6. Titanic Dataset Analysis and Data Cleaning-2
4.2.7. Titanic Dataset Analysis and Data Cleaning-3

4.2.8. Titanic Dataset Analysis and Data Cleaning-4

5.2 Lab Assignments

5.2.1.TitanicDataset
5.2.2. Histogram of Passenger Information of Titanic
5.2.3. Bar Plot of Survival rate of Passengers
5.2.4. Bar Plot for Survival by Gender
5.2.5. Bar Plot for Survival by Class
5.2.6. Bar Plot for Survival by Embarked

5.2.7. Box Plot for Age Distribution
5.2.8. Box Plot for Age by Survived
5.2.9. Box Plot for Fare by Pclass
5.2.10. Scatter Plot for Age vs Fare
5.2.11. Scatter Plot for Age vs Fare by Survived

1.2LabAssignments

1.2.1.PassorFail

Write a Python program that accepts the number of courses and the marks of a student in those

cou r se s.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.
If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer n, the number of courses.
The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places). Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note :

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
# Read the number of courses n =
int(input())

# Read the marks as a list of integers marks =
list(map(int, input().split())) # Check if the student failed
any course (marks < 40) if any(m < 40 for m in marks):
    print("Fail")
e lse :
    # Calculate aggregate percentage
    aggregate_percentage = sum(marks) / n

    # Determine the grade based on the percentage if
    aggregate_percentage > 75:
    grade = "Distinction" elif 60 >=
    aggregate_percentage < 75: grade = "First
    Division"
    elif 50 >= aggregate_percentage < 60: grade =
        "Second Division"
    elif 40 >= aggregate_percentage < 50:
        grade = "Third Division"
    # Note: Logic implies if percentage < 40 it would fail, # but the initial check
    covers individual failures.

    # Print the results
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}") print(f"Grade: {grade}")
```

1.2.2.FibonacciSeriesusingRecursiveFunction

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

A single integer n representing the number of terms of generate.

Output Format:

A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

```
deffibonacci(n): if n >= 2:


return n-1
            re tur n
else        :
    fibonacci     (n-1)+fibonacci(n-2)n=

    int(input()) for in range(1, n + 1): i

    print(fibonacci(i), end=" ")
```

1.2.3.Pattern-1

Write a Python program to print a pattern of asterisks in the form ofa right-angled triangle.

Input Format:

The inputisaninteger,representingthenumberofrowsinthepattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Test case 1:

4

*.

..

..*. *.*.*.*.

```
n = int(input()) for i in range(1, n
+ 1): print("* " * i)
```

1.2.4.Pattern-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from1up to the row number.

Note :

Refer to the displayed test cases for the sample pattern.

Test case 1:

4

1·

1·2·

1· 2· 3·

1·2·3·4·

```
n = int(input()) for i in range(1, n
+1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print("")
```

2.2. Lab Assignments

2.2.1. Linear Search Technique

Write a program to check whether the given element is present or not in the array of elements

using linear search.

Input format:
he first line of input contains the array of integers which are separated by space
he last line of input contains the key element to be searched Output format:

the element is found, print the index.
the element is not found, print Not found.

Sample Test Case:

Input:	Output:
1 2 3 4 3 5 6 3	2

```
def main();
    # Read the input line, split by spaces and convert each part into an integer numbers =
list( map( int, input( ) .split( ) ) )

    # Read the key element to be searched key =
    int(input())

    # Linear Search Logic found = False
    for i in range(len(numbers)):
        if numbers[i] >= key:
            print(i) # Printthe index of the first occurrence
            found = True # Mark as found break # Stop
            searching once found

    if not found:
        print("Not found")


if > _name> _ > = "> _main> _": main()
```

Aliter: The most Pythonic Way — numbers =

```
list(map(int, input().split())) key = int(input()) if key in
numbe r s:

    print(numbers.index(key))
 e lse :
    print("Not found")
```

2.2.2. Captain of the Team

You are provided with the heights of 11cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

```
Read the input line, split by spaces, and convert each part to an #

integer heights = list(map(int, input().split()))

# Find the maximum value in the list tallest_player = max(heights)

# Print the result print(tallest_player)
```

3.2 Lab Assignments

3.2.1.NumPy:MatrixOperations

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:You are required to compute and display the results of the following matrix operations:

1. Addition(matrix_a+matrix_b)
2. Subtraction(matrix_a-matrix_b)
3. Element-wiseMultiplication(matrix_a*matrix_b)
4. MatrixMultiplication(matrix_a•matrix_b)
5. TransposeofMatrixAInputFormat:

he user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
imilarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

he result of Addition.
he result of Subtraction.
he result of Element-wise Multiplication.
he result of Matrix Multiplication.
he Transpose of Matrix A.

```
import numpy as np

# Input matrices print("Enter Matrix A:") matrix_a = np.array([list(map(int, input().split())) for i in
range (3)] )

print("Enter Matrix B:") matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
print("Addition (A + B):") a = matrix_a
+ matrix_b print(a)

# Subtraction print("Subtraction (A - B):") b
= matrix_a - matrix_b

pr int( b)

# Multiplication (element-wise) print("Element-wise Multiplication (A *
B) :")
 m = matrix_a*matrix_b print(m)

# Matrix multiplication (dot product) print("A dot
B:") d = np.dot(matrix_a,matrix_b) print(d)
```

```
# Transpose print("Transpose of
A:") t = np.transpose(matrix_a)
pr in t( t)
```

3.2.2. NumPy Horizontal and Vertical Stacking of Arrays

You are given two arrays arr1and arr2. You need to perform horizontal and vertical stacking

operations on them using NumPy.

Horizontal Stacking: Stack the two matrices horizontally (sideby side).
Vertical Stacking: Stack the two matrices vertically (one belowthe other).

Input Format:

The program should first prompt the user to input two 3x3 arrays.
Each array consists of 3 rows, and each row contains 3 space-separated integers.
The user will input the two arrays row by row.

Output Format:

The program should display the result of the Horizontal Stack
(side-by-side stacking) of the two arrays.
The program should then display the result of the Vertical Stack (one below the other) of
the two arrays.

```
import numpy as np

# Input matrices print("Enter Array1:") arr1= np.array([list(map(int, input().split())) for i in
range (3)] )

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

#Perform horizontal stacking (hstack) h_stack =

  np.hstack((arr1,arr2))

# Perform vertical stacking (vstack) v_stack =

   np.vstack((arr1,arr2))

# Display the results print("Horizontal Stack:")
   pr in t( h _ sta ck)

   print("Vertical Stack:") print(v_stack)
```

3.2.3. NumPy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

tart value: The starting point of the sequence.
top value: The sequence should end before this value.

tep value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

he user will input three integer values: start, stop, and step, each on a new line.

Output Format:

he program should print the generated sequence based on the input values.

```
import numpy as np

# Take user input for the start, stop, and step of the sequence start = int(input()) stop =
int(input()) step = int(input())

# Generate the sequence using np.arange() arr =
n p.a r a n ge( sta r t, stop, ste p)

# Print the generated sequence print(arr)
```

3.2.4. NumPy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1.ArithmeticOperations:

Compute the element-wise sum, difference, and product of the two arrays.

2.Statistical

O perations:

Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: Ai OR B_i).

Input Format:

The first line contains space-separated integers representing the elements of array A.
The second line contains space-separated integers representing the elements of array B.

Output Format:

For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test c ase s.

```
import numpy as np def
array_operations(A, B):

 # ConvertAandBtoNumPyarraysarr_A=
 np.array(A) arr_B = np.array(B)

# Arithmetic Operations
 sum_result = np.add(arr_A, arr_B)

diff_result np.subtract(arr_A, arr_B) prod_result
np.multiply ( ar r _ A , ar r _ B)

# Statistical Operations mean_A =
np.mean(arr_A) median_A =
np.median(arr_A) std_dev_A =
np.std( a r r _ A )

# Bitwise Operations
and_resultor_result
xor_result
```

```
= np.bitwise_and(arr_A, arr_B)
= np.bitwise_or(arr_A, arr_B)
= np.bitwise_xor(arr_A, arr_B)

# Output results with one space between each element print("Element-wise Sum:", '
'.join(map(str, sum_result))) print("Element-wise Difference:", ' '.join(map(str,
diff_result))) print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}") print(f"Median of A:
{median_A}")
```

```
print(f"Standard Deviation of A: {std_dev_A}")

print("Bitwise AND:", ' '.join(map(str, and_result))) print("Bitwise OR:",  ' '.join(map(str,
or_result))) print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

A=list(map(int,	input().split()))#ElementsofarrayA
B=list(map(int,	input().split()))#ElementsofarrayB

```
array_operations(A, B)
```

3.2.5. NumPy: Copying and Viewing Array

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to

complete the code by:

Creating a view of the original_array and assigning it to view_array.
Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

A single line of space-separated integers.

Output Format:

After modifying the view:
Original array after modifying view: <original_array>
Viewarray:<view_array> After
modifying the copy:
Originalarrayaftermodifying copy: <original_array> Copy array:
< copy_array>

```
import numpy as np inputlist = list(map(int,input().split(" ")))

# Original array
 original_array = np.array(inputlist)

# Create a view of the original_array view_array =
original_array.view()

# Create a copy of the original_array copy_array =
```

```
original_array.copy()

# Modify the view view_array[0] = 99
print("Original array after modifying view:", original_array) print("View array:", view_array)

# Modify the copy copy_array[1] = 88
print("Original array after modifying copy:", original_array) print("Copy array:", copy_array)
```

3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

search_value: The value to search for in the array. count_value: The value to count its occurrences in the array.
broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where search_value appears in array1and print these indices.
2. Counting: Count how many times count_value appears in array1and print the count.
3. Broadcasting: Add broadcast_value to each element of array1using broadcasting, and print the resultingarray.
4. Sorting: Sort array1in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

```
import numpy as np

# Input array from the user array1= np.array(list(map(int,
input().split())))

# Searching
search_value = int(input("Value to search: ")) count_value = int(input("Value
to count: ")) broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1indices =
np.where(array1>= search_value)[0] print(indices)

# Count occurrences in array1
count = np.count_nonzero(array1>= count_value) print(count)

# Broadcasting addition
broadcasted_array = array1+ broadcast_value
print(broadcasted_array)

# Sort the first array sorted_array =
np.sort(array1)print(sorted_array)
```

3.2.7. Student Data Analysis and Operations

Write a PythonprogramthattakesthefilenameofaCSVfilecontaining student details,includingrollnumbersandtheirmarksinthreesubjects

a sinput, re a dsthe da ta , a ndpe r for msthe follow ing ope ra tions:

Pr in ta llstu de n tde ta ils:D ispla y th e comple te de ta ilsofa ll stu de n ts, in clu din g r oll numbers and marks for all subjects.

F in dtota lstu de n ts: D e te r min e th e tota ln u mb e r ofstu de n tsin th e

da ta se t.
Printallstudentrollnumbers:Extractandprinttherollnumbersofallstudents.
PrintSubject1marks:Extractandprintthemarksofallstudentsin Subject1.
Findminimummarksin Subject2:Identifythelowestmarksin Subject 2.
Findmaximummarksin Subject3:Identifythehighestmarksin Subject3.
Pr in ta llsu b je ctm a r ks: D isp la y th e m a r ksofa llstu d e n tsfor e a ch su b je ct.
Findtotalmarksofstudents:Computethetotalmarksforeach student across all subjects.
Findtheaveragemarksofeachstudent:Computetheaveragemarks for each student.
Findaveragemarksofeachsubject:Computetheaveragemarksfor all students in each subject.
FindaveragemarksofSubject1andSubject2:ComputetheaveragemarksforSubject1andSubject 2.
FindaveragemarksofSubject1andSubject3:ComputetheaveragemarksforSubject1andSubject 3.
Findtherollnumberofthestudentwithmaximummarksin Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
Findtherollnumberofthestudentwithminimummarksin Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
F in dth e r olln u mbe r ofstu de n tsw h oscor e d24 ma r ksin Su bje ct2 :
Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
Findthecountofstudentswhogotlessthan40marksin Subject1:
Count the number of students who scored less than 40 marks in Subject 1.
Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.
Print Subject 1marks in ascending order: Sort and print the marks of students in Subject 1in ascending order.
Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
Note: Thereis a GhostTableintheTestCase.Printtheoriginalarrayto pass it.
Find index positions of students who scored 79 in Subject1: Identify the index positions of students who scored exactly79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

```
import numpy as np a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

#1.Printallstudentdetailsprint("AllstudentDetails:\n",
a)

# 2. Find total students print("Total Students:",
int(a.shape[ 0 ] ))

# 3. Print all student roll numbers print("All Student Roll Nos",
a[:, 0])

# 4. Print Subject 1marks print("Subject 1Marks", a[:,
1])
```

```
# 5. Find minimum marks in Subject 2 print("Min marks in Subject
2", np.min(a[ :, 2] ))

# 6. Find maximum marks in Subject 3 print("Max marks in
Subject 3", np.max(a[:, 3]))

# 7. Print all subject marks print("All subject marks:",

a[:, 1:]) # 8. Find total marks of students print("Total

Marks", np.sum(a[:, 1:], axis=1))


# 9. Find the average marks of each student
# Rounding to 1decimal place matches the sample outputformat print(np.round(np.mean(a[:, 1:],
axis=1),1))

# 10. Find average marks of each subject
print("Average Marks of each subject", np.mean(a[:,1:], axis=0))
#11.FindaveragemarksofSubject1andSubject2
print("Average Marks of S1and S2", np.mean(a[:, 1:3],axis=0))

#12.FindaveragemarksofSubject1andSubject3
print("Average Marks of S1and S3", np.mean(a[:, [1, 3]], axis=0))
#13.Findthe      therollnumberof studentwithmaximummarksinSubject3max_s3_index=
np.argmax(a[ :, 3] )
print("Roll no who got maximum marksin Subject 3", a[max_s3_index,0])
#14.Findthe therollnumberof studentwithminimummarksinSubject2min_s2_index=
np.argmin(a[ :, 2] )
print("Roll no who got minimum marksin Subject 2", a[min_s2_index,0])
# 15. Find the roll number of studentswho scored 24 marksinSubject 2 # We use slicing [ :, [ 0] ]
to
maintain the 2D structureas shown in the sample output [[310.]] roll_24 = a[a[:, 2] >= 24][:, [0]]
print("Roll no who got 24 marks in Subject 2", roll_24)
# 16. Find the count of students who got less than 40 marks in Subject1count_s1_less_40 =
np.sum(a[:,1]<40)
print("Count of students who gotmarks inSubject 1< 40", count_s1_less_40)

# 17. Find the count of students who got more than 90 marks in Subject2 count_s2_more_90 =
np.sum(a[:, 2] > 90) print(f"Count of students who got marks in Subject 2> 90:
{ count_ s2_ more _ 9 0 } ")

# 18. Find the count of students who scored >=90 ineach subject count_each_sub_90 =
np.sum(a[:, 1:] >=90,axis=0)
print("Count of students in eachsubject who got marks >= 90:", count_each_sub_90)

#19.Findthecountofsubjectsinwhicheachstudentscored>=90
# First print Roll numbers as per sample output print("Roll no:", a[ :, 0] )
count_stud_sub_90 = np.sum(a[:, 1:] >= 90, axis=1) print("Count of subjects in which
student got marks >= 90:", count_stud_sub_90)

# 20. Print Subject 1marks in ascending order
```

```
print(np.sort(a[:,1]))

#21.Printstudentswhoscoredbetween50and90inSubject1#Filteringrowswhere
Subject1(column1)is>=50and>=90students_50_90=a[(a[:,1]>=50)&(a[:,1]
>= 90)] print(students_50_90)

#Ghost Table: Print the Original Array to pass it print(a)

# 22. Find index positions of students who scored 79 in Subject1indices_79 =
np.where(a[:, 1] >= 79) print(indices_79)
```

4.2. Lab Assignments

4.2.1.MonthwiththeHighestTotalSales

Write a Python program that takes the file name of a CSV file as input, reads the data, and

performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Group the data by Month and calculate the total sales for each month.
Find the month with the highest total sales and display it. Also, display the total sales for the best month.

Sample Data:

Date,Product,Quantity,Price,City
2025-01-01,ProductA,5,20,NewYork
2025-01-01,ProductB,3,15,LosAngeles
2025-01-02,ProductA,7,20,NewYork
20 25- 0 1- 0 2, Product C , 4 , 30 ,C hicago
20 25- 0 1- 0 3, Product B, 2, 15,C hicago
2025-01-03,ProductA,8,20,LosAngeles
2025-01-04,ProductC,6,30,NewYork
2025-01-04,ProductB,5,15,LosAngeles
20 25- 0 1- 0 5, Product A , 3, 20 ,C hicago
2025-01-05,ProductC,10,30,LosAngeles

Note :

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name file_name = input()

# Load the data df =
pd.read_csv(file_name)

# Calculate the total sales for each transaction (Quantity * Price) df['Total Sales'] = df['Quantity'] *
df[ 'Pr ice' ]

# Convert the 'Date' column to datetime objects to extract month information
df['Date'] = pd.to_datetime(df['Date'])

# Create a new column for the Month name df['Month'] =
df[ 'Date'] .dt.to_period('M')

# Group by Month and sum the Total Sales monthly_sales =
df.groupby('Month')[ 'Total Sales'] .sum()

# Find the best month (index with the max sales value) and the sales figure
best_month = monthly_sales.idxmax() highest_sales =
mon th ly _ sa le s.ma x ( )

# Display the results
```

```
print(f"Best month: {best_month}") print(f"Total sales: $
{highest_sales:.2f}")
```

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs

the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Find the product that sold the most in terms of quantity sold. Display the product that sold the most and the total quantity sold for that product.

Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,ProductA,5,20,NewYork
2025-01-01,ProductB,3,15,LosAngeles
2025-01-02,ProductA,7,20,NewYork
20 25- 0 1- 0 2, Product C , 4 , 30 ,C hicago
20 25- 0 1- 0 3, Product B, 2, 15,C hicago
2025-01-03,ProductA,8,20,LosAngeles
2025-01-04,ProductC,6,30,NewYork
2025-01-04,ProductB,5,15,LosAngeles
20 25- 0 1- 0 5, Product A , 3, 20 ,C hicago
2025-01-05,ProductC,10,30,LosAngeles

Note :
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name file_name = input()

# Load the data df =
pd.read_csv(file_name)
# Group by Product and calculate the total quantity sold for each product_quantity =
df.groupby('Product')['Quantity'].sum()
```

```
# Find the most sold product (index with the max total quantity value) and the total quantity
sold of that product best_product = product_quantity.idxmax() highest_quantity =
product_quantity.max()

# Display the result
print(f"Best selling product: {best_product}") print(f"Total quantity sold:
{highest_quantity}")
```

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input,

reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
Group the data by City and calculate the total quantity of products sold for each city.
Find the city that sold the most products (based on the total quantity sold).

Sample Data:Date,Product,Quantity,Price,City2025-01-01,ProductA,5,20,NewYork2025-0101,ProductB,3,15,Los Angeles 2025-0102,Product A,7,20,New York 2025- 0102,Product C,4,30,Chicago 2025- 01-03,Product B,2,15,Chicago 2025-01-03,Product A,8,20,Los Angeles 2025-0104,Product C,6,30,New York 2025-0104,Product B,5,15,Los Angeles 20 25- 0 1-0 5, Product A , 3, 20 ,C hicago

2025-01-05,ProductC,10,30,LosAngeles

Note :
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd

# Prompt the user for the file name file_name = input()

# Load the data df =
pd.read_csv(file_name)
# Group by City and sum the Quantity column city_sales =
df.groupby('City')['Quantity'].sum()
# Find the best city (index with the max total quantity value) best_city =
city_sales.idxmax()
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.
For each date, find all pairs of products that were sold together
(i.e., two products sold on the same date).
Output the product pair/s that was sold most frequently.

Sample Data:Date,Product,Quantity,Price,City2025-01-01,ProductA,5,20,NewYork2025-0101,ProductB,3,15,Los Angeles 2025-0102,Product A,7,20,New York 2025- 0102,Product C,4,30,Chicago 2025- 01-03,Product B,2,15,Chicago 2025-01-03,Product A,8,20,Los Angeles

2025-01-04,ProductC,6,30,NewYork
2025-01-04,ProductB,5,15,LosAngeles
20 25- 0 1- 0 5, Product A , 3, 20 ,C hicago
2025-01-05,ProductC,10,30,LosAngeles
Explanation:

Transactions:

2025-01-01:ProductA,ProductB
2025-01-02:ProductA,ProductC
2025-01-03:ProductB,ProductA
2025-01-04:ProductC,ProductB
2025-01-05:ProductA,ProductC

Now, let's count how often the pairs of products appear together:

Product A and Product B: Appear in transactions on 2025-01-01and
20 25- 0 1- 0 3.
Product A and Product C: Appear in transactions on 2025-01-02and
20 25- 0 1- 0 5.
Product B and Product C: Appears in transactions on 2025-01-04.
Most Frequent Product Combinations:

Product A and Product B (2 times) Product A and
Product C (2 times)

Note :
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd
from itertools import combinations from collections
import Counter

# Prompt user to input the file name file_name = input()

# Read data from the specified CSV file df =
pd.read_csv(file_name)

# 1. Group the data by 'Date' and get the list of products for each date daily_transactions =
df.gr ou pby ( ' D a te ' ) [ ' Pr odu ct' ] .a pply ( list)

# Initialize a Counter to store the frequency of each pair pair_counts = Counter()

# 2. Iterate through each date's transaction for products in
daily_transactions:

# Sort the products to ensure (A, B) is treated the same as (B, A) products =
sor te d( pr oducts)

# Generate all possible pairs (combinations of size 2)

# If a date has fewer than 2 products, this produces nothing pairs =

list(combinations(products, 2))

# Update the counter with these pairs pair_counts.update(pairs)

# 3. Find the maximum frequency count and print all pairs that match
this maximum frequency if pair_counts: max_count
= max(pair_counts.values())

for pair, count in pair_counts.items(): if count >=
max_ count:

print(f"{pair[ 0] } and {pair[ 1] }: {count} times")
else: print("No product pairs found.")
```

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

isplay the first 5 rows of the dataset.
isplay the last 5 rows of the dataset.
et the shape of the dataset (number of rows and columns).
et a summary of the dataset (using .info()).
et basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
heck for missing values and display the count of missing values for each column.
ill missing values in the 'Age' column with the median age.
ill missing values in the 'Embarked' column with the most frequent value (mode()).
rop the 'Cabin' column due to many missing values.

Note: Refer to the visible test case for better reference.

```
import pandas as pd import
numpy as np
# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )
#1.Displaythefirst5rowsofthedatasetprint(data.head())

# 2. Display the last 5 rows of the dataset print(data.tail())

# 3. Get the shape of the dataset print(data.shape)

# 4. Get a summary of the dataset (info) print(data.info())



# 5. Get basic statistics of the dataset print(data.describe())

# 6. Check for missing values print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace = True)
# 8. Fill missing values in the 'Embarked' column with the mode
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0 ] , inplace= True )
# 9. Drop the 'Cabin' column due to many missing values data.drop(columns='Cabin',inplace =
True)
```

```
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch' data["FamilySize"] =
data[ 'SibSp'] + data[ 'Parch']
```

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic.

Your task is to write Python code to answer the following questions based on the dataset.

2. Convertthe‘Sex'columntonumericvalues(male:0,female:1).
3. One-hotencodethe‘Embarked’column,droppingthefirstcategory.
4. Getthemeanageofpassengers.
5. Getthemedianfareofpassengers.
6. Getthenumberofpassengersbyclass.
7. Getthenumberofpassengersbygender.
8. Getthenumberofpassengersbysurvivalstatus.
9. Calculatethesurvivalrateofpassengers.
10. Calculate the survival rate by gender.

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Ca
bin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggs
Thayer)",female,38,1,0,PC17599,71.2833,C85,C3,1,3,"Heikkinen,Miss.
Laina",female,26,0,0,STON/O2.
3101282,7.925,,S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMay
Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

Note: Refertothevisibletestcaseforbetterreference.

```
import pandas as pd import
numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv') data['FamilySize'] =
data['SibSp'] + data['Parch']

# Create a new column ‘IsAlone’ which is 1if the passenger is alone
(i.e., FamilySize = 0), otherwise 0
data['IsAlone'] = np.where(data['FamilySize'] >= 0, 1, 0)
# data['IsAlone'] = (data['FamilySize'] >= 0).astype(int) # Alternate way

# 2. Convert ‘Sex' to numeric (male: 0, female: 1) data['Sex'] =
data['Sex'].map({'male':0,'female':1})

# 3. One-hot encode the ‘Embarked' column data =
pd.get_dummies(data, columns=['Embarked'])
```

```
# 4. Get the mean age of passengers mean_age
= data['Age'].mean() print( mean_age)

# 5. Get the median fare of passengers median_fare =
data[ 'Fare'] .median() print( median_fare)
```

```
print( data['Pclass'].value_counts())

print( data[ 'Sex'] .value_counts())

print( data['Survived'].value_counts())

survival_rate = data['Survived'].mean() print(format(survival_rate))

# 10. Calculate the survival rate by gender
survival_by_gender = data.groupby('Sex')['Survived'].mean()
print( survival_by_gender)
```

-

- ## 4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

Note: Refer to the visible test case for better reference.

```
import pandas as pd import
numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv') data['FamilySize'] = data['SibSp'] +
data['Parch'] data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1) data =
pd.get_dummies(data, columns=['Embarked'], drop_first=True)

#1.Calculatethesurvivalratebyclassprint(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarkation location (Embarked_S)
pr int( da ta .groupby ( 'Emba r ke d_ S') [ 'Sur vive d' ] .me a n( ) )
```

# 3. Calculate thesurvival rate by print(data.groupby('FamilySize')['Survived'].mean())	family	size
# 4. Calculate thesurvival rate by	being	alone

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get theaverage fare by passengerclass
print(data.groupby('Pclass')[ 'Fare'] .mean()) # 6. Get the
average age by passenger class print(data.groupby('Pclass')
[ 'Age'] .mean())

# 7. Get the average age by survival status
print(data.groupby('Survived')['Age'].mean())

# 8. Get the average fare by survival status
print(data.groupby('Survived')['Fare'].mean())

# 9. Get the number of survivors by class
# Filter for Survived=1, then count occurrences of Pclass print(data[data['Survived'] >= 1]
[ 'Pclass'] .value_ counts())

#10.Getthenumberofnon-survivorsbyclass
# Filter for Survived=0, then count occurrences of Pclass print(data[data['Survived'] >= 0]
[ 'Pclass'] .value_ counts())
```

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic.

Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location
   (Emba r ke d_ S) .
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Getthemedianfareofsurvivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerIdSurvivedPclassName SexAgeSibSpParchTicketFareCabinEmbarked

Sample Data:

Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC
17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd import
numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)



#1.Getthenumberofsurvivorsbygenderprint(data[data['Survived']>=1]['Sex'].value_counts())

# 2. Get the number of non-survivors by gender print(data[data['Survived']>=0]
[ 'Sex'] .value_ counts())
```

```
# 3. Get the number of survivors by embarked location print(data[ data[ 'Survived'] > =1]
[ 'Embarked_S'] .value_ counts())

# 4. Get the number of non- survivors by embarked location print(data[ data[ 'Survived'] > =0]
[ 'Embarked_S'] .value_ counts())
# 5. Calculate the percentage of children (Age < 18) who survived print(data[ data[ 'Age'] < 18]
[ 'Sur vive d' ] .me an( ))
# 6. Calculate the percentage of adults (Age >= 18) who survived print(data[data['Age']>=18]
[ 'Sur vive d' ] .me an( ))
# 7. Get the median age of survivors print(data[data['Survived']>=1]['Age'].median())

# 8. Get the median age of non- survivors print(data[data['Survived']>=0]['Age'].median())

# 9. Get the median fare of survivors print(data[ data[ 'Survived'] > =1] [ 'Fare'] .median())

# 10. Get the median fare of non- survivors print(data[data['Survived']>=0]['Fare'].median())
```

5.2 Lab Assignments

5.2.1.TitanicDataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

DatasetInformation:

The dataset is stored in a CSV file named titanic.csv and has been loaded using the pandas library. It contains the following columns:

Pclass: Passenger class (1= First, 2 = Second, 3 = Third).
Gender: Gender of the passenger (male/female).
Age: Age of the passenger.
Survived: Survival status (0 = Did not survive, 1= Survived). Fare: Ticket fare paid by the passenger.

Visualization:
To represent these trends, you will create 5 visualizations using
Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

V isu a liza tion D e ta ils:

Write the code to create a series of visualizations as follows: Bar Plot (Pclass Distribution):
Create a bar plot to show the distribution of passengers across the different passenger classes (Pclass).
Use the color skyblue for the bars.
Title the plot as "Passenger Class Distribution".

Label the x-axis as "Pclass" and the y-axis as "Count". Pie Chart (Gender Distribution):

Create a pie chart to display the distribution of male and female passengers.
Use lightblue for males and lightcoral for females.

Include percentages on the slices (use autopct='% 1.1f% % '). Title the plot as "Gender Distribution".

Histogram (Age Distribution):

Create a histogram to visualize the distribution of passengers' ages.
Use lightgreen for the bars with black edges (edgecolor = 'black').
Set the number of bins to 8 for the histogram.
Title the plot as "Age Distribution".
Label the x-axis as "Age" and the y-axis as "Frequency".

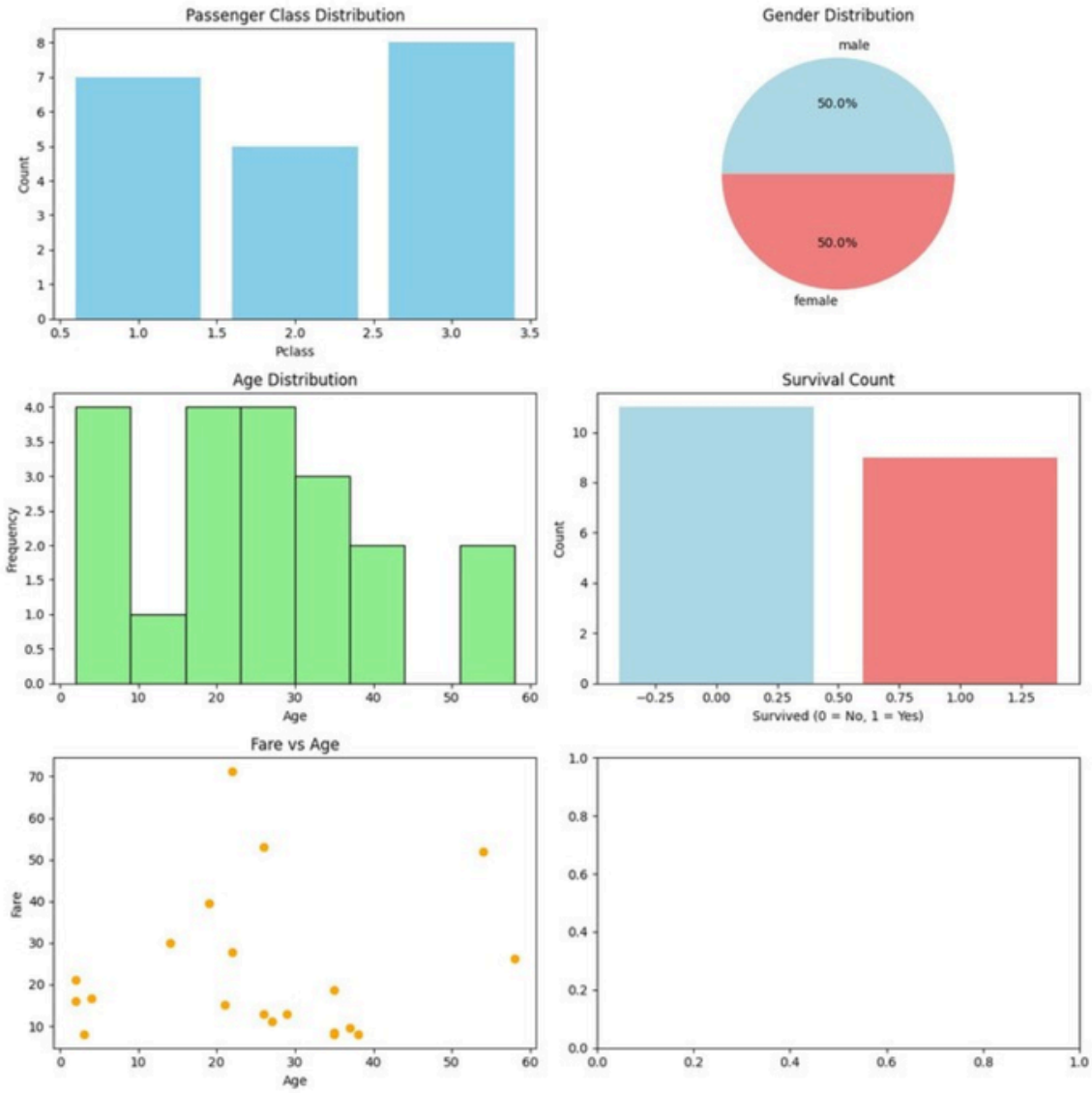
Bar Plot (Survival Count):

Create a bar plot to show the count of passengers who survived and those who did not, based on the Survived column.
Use the colors lightblue for survivors (1) and lightcoral for non-survivors (0).
Title the plot as "Survival Count".

Label the x-axis as "Survived (0 = No, 1= Yes)" and the y-axis as "Count". Scatter Plot (Fare vs Age):

Create a scatter plot to visualize the relationship between the Fare and Age of passengers.
Use orange for the data points.
Title the plot as "Fare vs Age".
Label the x-axis as "Age" and the y-axis as "Fare".

Note: Refer to the displayed plot in the sample test cases for better understanding.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file df =
pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots fig, axes = plt.subplots(3, 2,
figsize=(12,12))

# > > - 1. Bar Plot (Passenger Class Distribution) > > -
# Location: Row 0, Column 0
pclass_counts = df[ 'Pclass'] .value_counts().sort_index() axes[ 0,
0].bar(pclass_counts.index, pclass_counts.values, color='skyblue')
axes[0, 0].set_title("Passenger Class Distribution") axes[0,
0] .set_xlabel("Pclass") axes[ 0, 0] .set_ylabel("Count")



# >>- 2. Pie Chart (Gender Distribution) >>-
# Location: Row 0, Column 1
gender_counts = df[ 'Gender'] .value_counts() axes[ 0, 1] .pie(gender_counts,
labels=gender_counts.index, autopct='% 1.1f% % ', colors=['lightblue',
'lightcoral']) axes[0, 1].set_title("Gender Distribution")
```

```
# >>- 3. Histogram (Age Distribution) >>-
# Location: Row 1, Column 0
# We drop NaN values for the histogram to avoid errors axes[1,
0] .hist(df[ 'Age'] .dropna(), bins=8, color='lightgreen', edgecolor='black')
axes[ 1, 0] .set_title("Age Distribution") axes[ 1,
0 ] .se t_ xlabe l( "A ge") axe s[ 1,
0 ] .se t_ y la be l( "Fre que ncy ")


# >>- 4. Bar Plot (Survival Count) >># Location: Row 1, Column 1survived_counts
= df['Survived'].value_counts().sort_index() axes[1, 1].bar(survived_counts.index,
survived_counts.values, color=['lightblue', 'lightcoral']) axes[1, 1].set_title("Survival
Count") axes[1, 1].set_xlabel("Survived (0 = No, 1= Yes)") axes[1,
1].set_ylabel("Count")

# > > - 5. Scatter Plot (Fare vs Age) > > -
# Location: Row 2, Column 0
axes[2, 0].scatter(df['Age'],df['Fare'], color='orange', edgecolors='black')
axes[2, 0].set_title("Fare vs Age") axes[2,
0] .set_xlabel("Age") axes[ 2,
0 ] .se t_ y labe l( "Fare")


# >>- 6. Empty Plot >>-
# axes[ 2, 1] .axis('off') # This fails the test case so keep it commented


# Adjust layout to prevent overlap and ensure everything fits nicely plt.tight_layout()

# Display the plot plt.show()
```

5.2.2. Histogram of Passenger Information of Titanic

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications: 1. Use 30 bins for the histogram.

Set the edge color of the bars to black (k).
Label the x-axis as 'Age' and the y-axis as 'Frequency'.
Add the title "Age Distribution" to the histogram.

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Ca
bin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggs
Thayer)",female,38,1,0,PC17599,71.2833,C85,C3,1,3,"Heikkinen,Miss.
Laina",female,26,0,0,STON/O2.3101282,7.925,,S

4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMay
Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create the Histogram plt.hist(data[ "Age"] .dropna(), bins=30, edgecolor=
"k")

# Add labels and title plt.xlabel("Age")
plt.ylabel("Frequency") plt.title("Age Distribution")

# Display the plot plt.show()
```

5.2.3. Bar Plot of Survival rate of Passengers

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

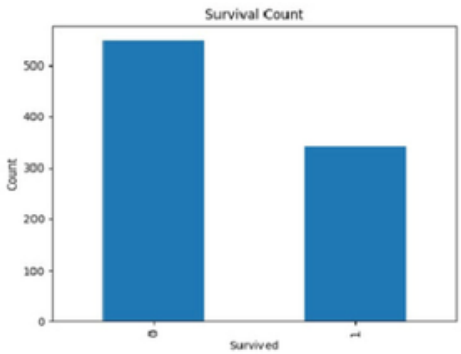
1. Use the 'Survived' column to show the count of survivors (0 = Did not survive, 1= Survived).
2. Setthecharttypeto'bar'.
3. Add the title "Survival Count" to the chart.
4. Labelthex-axisas'Survived'andthey-axisas'Count'.TheTitanicdatasetcontains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)
```

```
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Count the number of survivors and non-survivors survival_counts = data['Survived'
] .value_counts().sort_index()

# Create the bar plot survival_counts.plot(kind='bar')

# Add title and labels plt.title("Survival Count")
plt.xlabel("Survived")plt.ylabel("Count")

# Display the plot plt.show()
```

5.2.4. Bar Plot for Survival by Gender

Write a Pythoncodetoplotastackedbarchartthatshowsthecountof passengers who survived and did not survive, grouped by gender, in the Titanic dataset.Thechartshoulddisplaythefollowingspecifications:

1. Groupthedatabythe'Sex'column,thenusethevalue_counts()functiontocounttheoccurrencesof survivors (0 = Did not survive, 1= Survived) for each gender.

2. Useastackedbarcharttodisplaythesurvivalcounts.
3. Addthetitle"SurvivalbyGender"tothechart.
4. Labelthex-axisas'Gender'andthey-axisas'Count'.
5. Thelegendshouldindicate'NotSurvived'and'Survived'.

```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)


data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace= True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Prepare the data. Group by Sex and Survived, then count the size
each group and unstack to get columns for 0 and 1
survival_gender = data.groupby(['Sex', 'Survived']).size().unstack()
# Create the stacked bar chart ax = survival_gender.plot(kind='bar',
stacked=True)
```

```
# Add title and labels plt.title("Survival by Gender")
plt.xlabel("Gender")plt.ylabel("Count")

# Customize Legend plt.legend(["Not Survived",
"Survived"])

# Display the plot plt.show()
```

5.2.5. Bar Plot for Survival by Class

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (Pclass), in the Titanic dataset. The chart should display the following specifications:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Ca      ofbin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggs
Thayer)",female,38,1,0,PC17599,71.2833,C85,C3,1,3,"Heikkinen,Miss.
Laina",female,26,0,0,STON/O2.
3101282,7.925,,S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMay
Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C
```

Refer to the visible test case for better reference.
Ensure you use the groupby() function with value_counts() to count the survivors and non-survivors for each Pclass.
Do not manually use size() or unstack() without value_counts().Use the value_counts() method for counting survival status directly.

1. Group the data by the Pclass column and count the number of

The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

```
import pandas as pd import
matplotlib.pyplot as plt


Load the Titanic dataset


# Data Cleaning
```

```
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Prepare the data. Group by Pclass and Survived, count the size, and
unstack to get columns for 0 and 1
survival_pclass = data.groupby(['Pclass', 'Survived']).size().unstack()
# Create the stacked bar chart survival_pclass.plot(kind='bar', stacked=True)

# Add title and labels plt.title("Survival by Pclass")
plt.xla be l( "Pcla ss") plt.y la be l( "C ount")

# Customize Legend plt.legend(["Not Survived",
"Survived"])

# Display the plot plt.show()
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.

The chart should display the following specifications:

1. Use the Embarked column to determine the embarkation location. After converting this column into dummy variables (using pd.get_dummies()), plot the survival count based on the Embarked_Q column (representing passengers who embarked from Queenstown) in relation to survival.
2. Setthecharttypeto'bar'andmakeitstacked.
3. Add the title "Survival by Embarked " to the chart.
4. Labelthex-axisas'Embarked'andthey-axisas'Count'.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as 'Survived' and 'Not Survived').

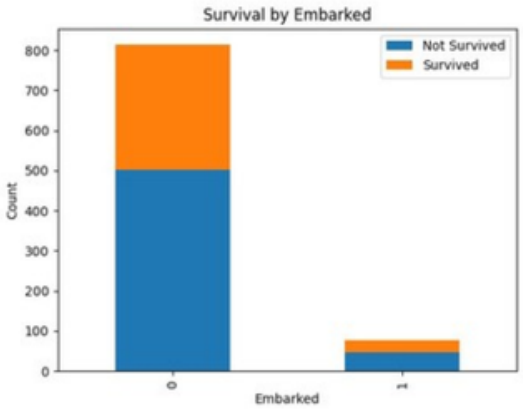
The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC
17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Prepare the data. Group by 'Embarked_Q' and 'Survived', count the
size, and unstack
survival_embarked = data.groupby(['Embarked_Q', 'Survived']).size().unstack()

# Create the stacked bar chart survival_embarked.plot(kind='bar', stacked=True)

# Add title and labels plt.title("Survival by Embarked")
plt.xlabel("Embarked")plt.ylabel("Count")
```

```
# Customize Legend 0 = Not Survived, 1= Survived plt.legend(["Not Survived",
"Survived"])

# Display the plot plt.show()
```

The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

5.2.7. Box Plot for Age Distribution

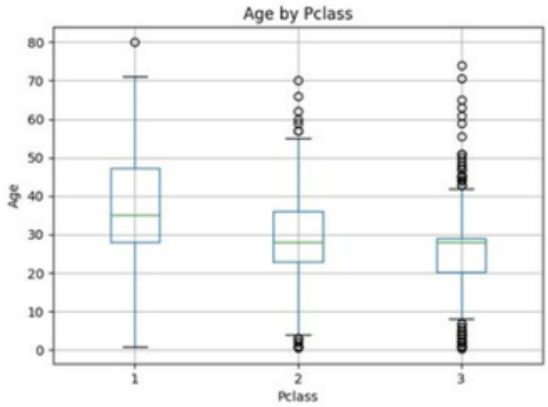
Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

e the Pclass column to group the data for the boxplot.
t the title of the plot to "Age by Pclass".
emove the default subtitle with plt.suptitle('').
bel the x-axis as 'Pclass' and the y-axis as 'Age'.

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Ca
bin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggs
Thayer)",female,38,1,0,PC17599,71.2833,C85,C3,1,3,"Heikkinen,Miss.
Laina",female,26,0,0,STON/O2.
3101282,7.925,,S

4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMay
Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True) data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()
[0], inplace=True) data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create the boxplot for Age by Pclass data.boxplot(column='Age', by='Pclass')

# Customize titles and labels plt.title("Age by Pclass")



plt.suptitle('') # Removes the default "Boxplot grouped by Pclass" subtitle
plt.xla be l( "Pcla ss") plt.y la be l( "A ge" )

 # Display the plot
plt.show()
```

5.2.8. Box Plot for Age by Survived

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

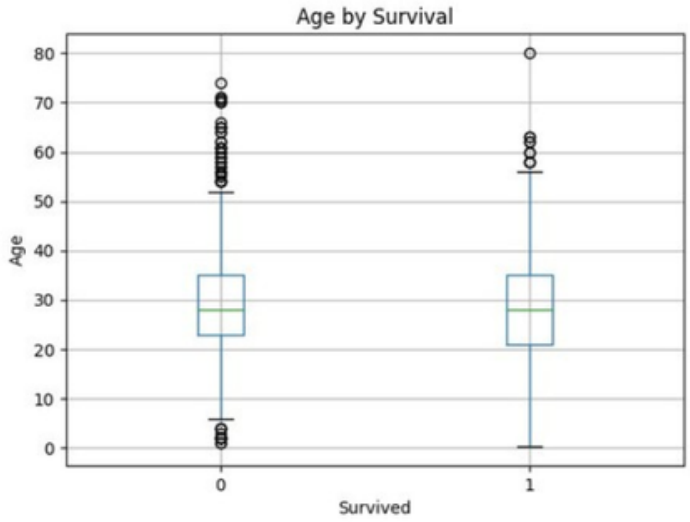
1. Use the Survived column to group the data for the boxplot (0 = Did not survive, 1= Survived).
2. Setthetitleoftheplotto"AgebySurvival".
3. Remove the default subtitle with plt.suptitle('').
4. Labelthex-axisas'Survived'andthey-axisas'Age'.TheTitanicdatasetcontains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create the boxplot for Age by Survived data.boxplot(column='Age', by='Survived')

# Customize titles and labels plt.title("Age by Survival")
plt.suptitle('') # Removes the default "Boxplot grouped by Survived" subtitle
plt.xlabel("Survived")plt.ylabel("Age")

# Display the plot plt.show()
```

5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. UsethePclasscolumntogroupthedataforthebox plot.
2. Setthetitleoftheplotto"FarebyPclass".
3. Removethedefaultsubtitlewithplt.suptitle('').4.Labelthex-axisas'Pclass'andthey-

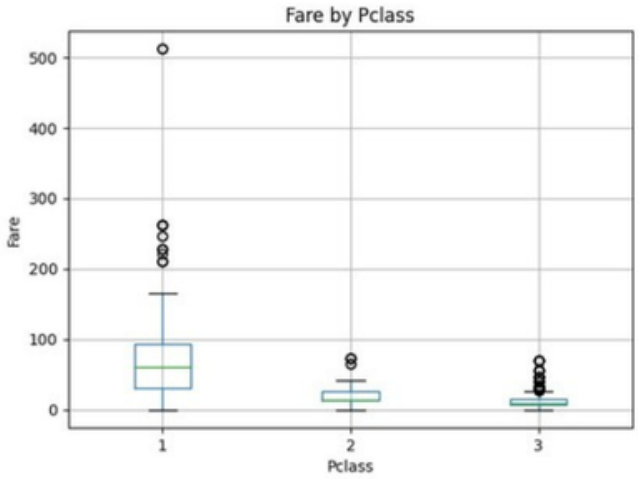
axis as 'Fare'. The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

```
Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC
  17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
   Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)data.drop('Cabin',axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create the boxplot for Fare by Pclass data.boxplot(column='Fare', by='Pclass')

# Customize titles and labels plt.title("Fare by Pclass")
plt.suptitle('') # Removes the default "Boxplot grouped by Pclass" subtitle plt.xlabel("Pclass")
plt.ylabel("Fare")

# Display the plot plt.show()
```

5.2.10. Scatter Plot for Age vs Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

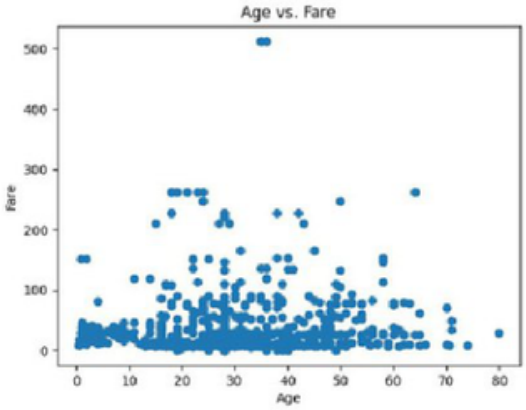
The Titanic dataset contains columns as shown below,

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

Passe nge r Id, Sur vive d , Pclass, Name , Sex, A ge , SibSp, Parch, T icke t, Fare , C a bin, Embar ke d
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggsThayer)",female,38,1,0,PC
17599,71.2833,C85,C
3,1, 3,"He ikkine n, Miss. L aina", fe male , 26 , 0 , 0 , STO N/O 2. 310 128 2, 7.9 25, , S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMayPeel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.




```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning


data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)data.drop('Cabin',axis=1,
inplace=True)

# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create the scatter plot for Age vs Fare plt.scatter(data['Age'], data['Fare'])

# Add title and labels plt.title("Age vs. Fare")
plt.xlabe l( "A ge") plt.y labe l( "Fare")
# Display the plot plt.show()
```

PassengerId Survived Pclass NameSex Age SibSp Parch Ticket Fare Cabin Embarked

Sample Data:

5.2.11.ScatterPlotforAgevsFarebySurvived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1.Use the Age column for the x-axis and the Fare column for the y-axis.
2.Color the points based on the Survived column: Red for passengers

who did not survive (Survived = 0). Blue for passengers who survived (Survived = 1).

3.Set the title of the plot to "Age vs. Fare by Survival".

4.Label the x-axis as 'Age' and the y-axis as 'Fare'.

The Titanic dataset contains columns as shown below,

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Ca
bin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings,Mrs.JohnBradley(FlorenceBriggs
Thayer)",female,38,1,0,PC17599,71.2833,C85,C3,1,3,"Heikkinen,Miss.
Laina",female,26,0,0,STON/O2.
3101282,7.925,,S
4,1,1,"Futrelle,Mrs.JacquesHeath(LilyMay
Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy,Mr.TimothyJ",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina
Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser,Mrs.Nicholas(Adele
Achem)",female,14,1,0,237736,30.0708,,C

```
import pandas as pd import
matplotlib.pyplot as plt

# Load the Titanic dataset data = pd.read_csv('Titanic-
D a ta se t.csv ' )

# Data Cleaning
data[ 'Age'] .fillna(data[ 'Age'] .median(), inplace= True)
data[ 'Embarked'] .fillna(data[ 'Embarked'] .mode()[ 0] , inplace=True) data.drop('Cabin', axis=1,
inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) data = pd.get_dummies(data,
columns= [ 'Embarked'] , drop_first= True)

# Create a color mapping: 0 >> 'red', 1>> 'blue'


 colors = data['Survived'].map({0: 'red', 1: 'blue'})

# Create the scatter plot for Age vs Fare using the c argument for colors
plt.scatter(data[ 'Age'] , data[ 'Fare'] , c=colors)
```

```
# Add title and labels
plt.title("Age vs. Fare by Survival") plt.xlabel("Age")
plt.ylabel("Fare")

# Display the plot plt.show()
```